Introduction to Integer Linear Programming

BUS 104: Decision Analysis and Management Science

Danko Turcic

November 9, 2026

UNIVERSITY OF CALIFORNIA
UC RIVERSIDE School of Business
A. GARY ANDERSON GRADUATE SCHOOL OF MANAGEMENT

Agenda: Integer Linear Programs Introduced in Class

Welcome! In this deck, we formulate and solve ten foundational **Integer (ILP)** and **Mixed-Integer (MILP)** models in `PuLP 4.0`:

1. **Pure ILP & Branch-and-Bound:** Discrete production, why LP rounding fails, and exact tree search.
2. **0–1 Knapsack Problem:** Binary take-or-leave selection ($x_i \in \{0,1\}$) under a capacity limit vs. greedy heuristics.
3. **Knapsack with Logical Constraints:** Conditional inclusion ($x_1 \le x_2$), mandatory selection ($x_1 = 1$), mutual exclusion ($x_1 + x_2 \le 1$), and XOR ($x_1 + x_2 = 1$).
4. **Fixed-Charge Production (MILP):** Activating a facility at a fixed setup cost using a Big-M linking constraint ($x_1 + x_2 \le My$).
5. **Minimum Purchase Quantity / Threshold Problem (MILP):** Enforcing order thresholds via two-sided switching constraints ($6y \le x_1 + x_2 \le 16y$).
6. **Fixed-Charge Transshipment (MILP):** Joint warehouse selection ($y_i \in \{0,1\}$) and multi-echelon network flow conservation.
7. **Store Location / Set Covering (Binary ILP):** Covering all city grid sectors with the minimum number of stores ($\sum_{j \in N(i)} x_j \ge 1$).
8. **Last-Mile Delivery Routing & The Subtour Trap (0–1 TSP):** Routing a delivery van across customer stops and eliminating disconnected subtour loops.
9. **Sports Analytics: Fantasy Lineup Optimization (0–1 ILP):** Building a salary-cap starting lineup with position quotas, teammate stacking, and locker-room chemistry rules.
10. **Currency Arbitrage & Profitable FX Loops (0–1 Tour ILP):** Making a multi-currency tour from USD back to USD using log-exchange-rates to capture risk-free arbitrage profit.

What is Integer Linear Programming?

Linear Program (LP)

- Optimization problem where the objective function and constraints are linear.
- All decision variables (x_i) are **continuous** (real) values.
- Solution methods (e.g., Simplex) are highly efficient in practice.

Integer Linear Program (ILP)

- An LP with the added restriction that some/all variables are **integers**.
- If all variables are integers, it is a *pure ILP*.
- Ideal for discrete choices (yes/no, counts).
- Simplex alone cannot solve them (requires Branch-and-Bound).

Key Difference: Variable Domain Restriction ($\mathbb{R}$ vs. $\mathbb{Z}$)

A Simple Pure ILP Formulation

Consider a two-variable problem (e.g., producing two products x_1, x_2) subject to resources.

Objective Function

Maximize profit:

$$\max z = 3x_1 + 2x_2$$

Constraints

$$\begin{aligned} (C_1) \quad & 2x_1 + x_2 \leq 7 \\ (C_2) \quad & -x_1 + x_2 \leq 3 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Integer Requirement (ILP)

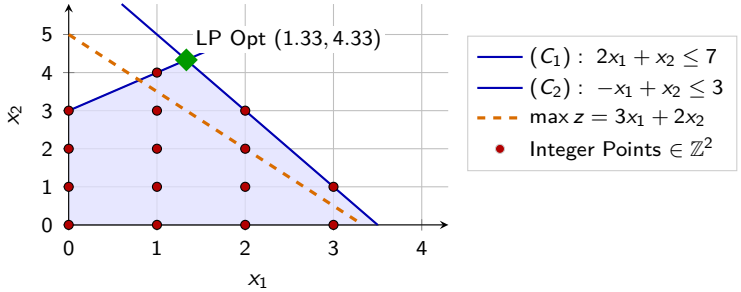
All variables must be integers:

$$x_1, x_2 \in \mathbb{Z}_{\geq 0}$$

LP Relaxation

- First solve assuming continuous $x_1, x_2 \in \mathbb{R}_{\geq 0}$.
- The LP optimal solution can be determined graphically or using Excel or Python via the Simplex algorithm.

Visualizing the Feasible Space

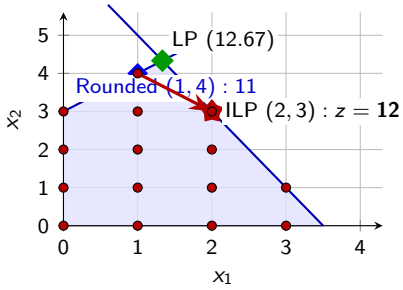


- The shaded region is the continuous LP feasible space; red dots (•) are the 15 integer-feasible solutions.
- **LP Optimum (◊):** $(x_1^*, x_2^*) = (4/3, 13/3)$ with $z^* = 12.67$.
- The LP optimal solution can be determined graphically or using Excel or Python via the Simplex algorithm.

Why the LP Solution is Not Enough

The common mistake is to simply round the fractional LP solution to the nearest integer point.

- **LP Optimum:** $(4/3, 13/3) \approx (1.33, 4.33)$, $z \approx 12.67$.
- **Nearest Integer Points (Rounding):**
  - Down-round: $(1, 4)$, $z = 3(1) + 2(4) = \mathbf{11}$ (feasible).
  - Up-round: $(2, 5)$ (infeasible: $2(2) + 5 = 9 \not\leq 7$).
  - Mixed: $(2, 4)$ (infeasible: $2(2) + 4 = 8 \not\leq 7$).



Rounding can be feasible but sub-optimal.

True ILP Optimum:

- Search the feasible integer points.
- Optimum at $\star$: $(2, 3)$, $z = 3(2) + 2(3) = \mathbf{12}$.

Conclusion: The true ILP optimum ($\star$) beats the rounded LP point, requiring systematic ILP algorithms.

Solving the Pure ILP in Python (PuLP 4.0)

Example 1: Continuous LP Relaxation vs. Pure ILP

```python
import pulp

# 1. Initialize Pure ILP Model (PuLP 4.0 API)
prob = pulp.LpProblem("Simple_Pure_ILP", pulp.LpMaximize)

# 2. Integer Decision Variables: x1, x2 in Z_{>= 0}
x1 = prob.add_variable("x1", lowBound=0, cat=pulp.LpInteger)
x2 = prob.add_variable("x2", lowBound=0, cat=pulp.LpInteger)

# 3. Objective Function
prob += 3 * x1 + 2 * x2, "Total_Profit"

# 4. Resource Constraints
prob += 2 * x1 + x2 <= 7, "C1"
prob += -x1 + x2 <= 3, "C2"

# 5. Solve and inspect Result
res = prob.solve()
print(f"Status: {res.status.name}")
print(f"Optimal z* = {pulp.value(prob.objective):.2f}")
print(f"x1* = {x1.varValue}, x2* = {x2.varValue}")
```

Output Comparison

- **LP Relaxation (`LpContinuous`):**
  - $x_1^* = 1.33,\ x_2^* = 4.33$
  - $z_{\text{LP}}^* = 12.67$
- **Pure ILP (`LpInteger`):**
  - `Status: Optimal`
  - $x_1^* = 2.0,\ x_2^* = 3.0$
  - $z_{\text{ILP}}^* = \mathbf{12.00}$

PuLP 4.0 Tip

Use `prob.add_variable(..., cat=pulp.LpInteger)` to require whole-number (integer) values and `res = prob.solve()` to capture the solution status object.

Section 1

Branch and Bound Method

Branch-and-Bound (B&B): The Idea

Why B&B?

- LP relaxation is fast but may yield fractional values.
- ILP requires whole numbers; simple rounding can be suboptimal or infeasible.

Core Mechanics

1. **Relax** integer rules (allow fractions) and solve LP $\Rightarrow$ upper bound (UB).
2. **Branch** on a fractional variable ($x_i \leq \lfloor x_i^* \rfloor$ vs. $x_i \geq \lceil x_i^* \rceil$).
3. **Bound:** prune subproblems whose LP bound $\leq$ best known integer solution (incumbent), or infeasible.
4. Continue until all nodes are pruned or integer.

Goal: Prove optimality by excluding all non-promising regions.

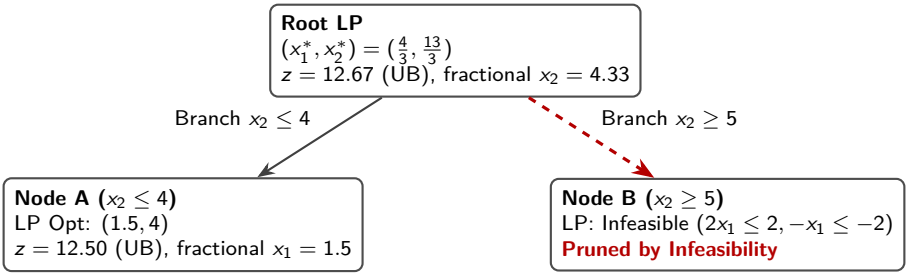
Our ILP (Same as Before)

$$\max z = 3x_1 + 2x_2$$

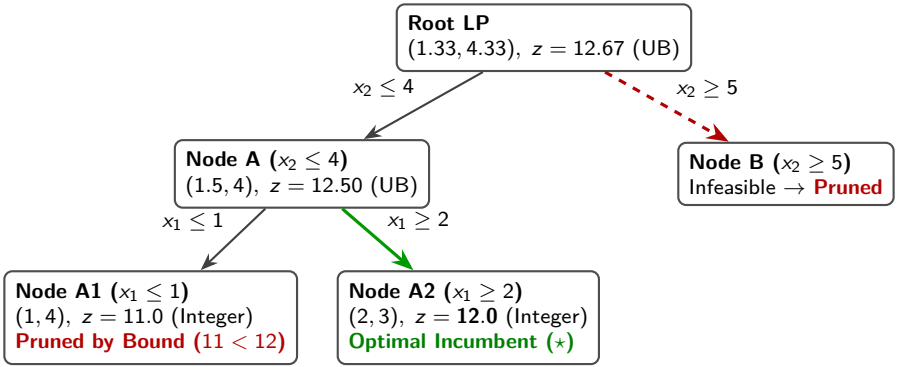
$$2x_1 + x_2 \leq 7,$$
$$-x_1 + x_2 \leq 3,$$
$$x_1, x_2 \geq 0, \quad x_1, x_2 \in \mathbb{Z}.$$

Root LP Relaxation Optimum:
$(x_1^*, x_2^*) = \left(\frac{4}{3}, \frac{13}{3}\right), \quad z^* \approx 12.67$ (fractional).

B&B Step 1: Branch on the Fractional Variable



B&B Step 2: Complete Search Tree and Bounds



Section 2

The 0-1 Knapsack Problem

Problem Definition: The 0-1 Knapsack

What is the 0-1 Knapsack Problem?

The Classic Analogy

Imagine you have a knapsack (or bag) with a limited weight capacity, W.

- You are presented with n different items.
- Each item i has:
  - A **value**, v_i
  - A **weight**, w_i
- You must make a binary ("0-1") decision for each item:
  - Either you **take** the item ($x_i = 1$)
  - Or you **leave** the item ($x_i = 0$)
- You cannot take fractions of items.

The Goal

Choose the combination of items that **maximizes the total value** in your knapsack...

...without **exceeding the total weight capacity** W.

Why Do We Obsess Over the 0-1 Knapsack?

From Hiking Backpacks to High-Stakes Business Decisions

Core Idea: Managers rarely pack physical backpacks, but constantly make binary yes/no decisions ($x_i \in \{0, 1\}$) to maximize total value (v_i) under a strict capacity or budget limit (W).

Spotlight: Revenue Management at Amazon (Personalized Search Assortment)

- When you search on Amazon, thousands of products match, but **screen space is limited (W)**.
- Using your browsing and purchase history, Amazon estimates the **expected value/revenue (v_i)** of showing you each product i, while each listing consumes **screen space (w_i)**.
- Amazon solves a 0-1 Knapsack problem in milliseconds to decide which products to show ($x_i = 1$) vs. hide ($x_i = 0$).

Application	Decision ($x_i \in \{0, 1\}$)	Capacity (W)	Weight (w_i)	Value (v_i)
Amazon Search / Retail RM	Show product i?	Screen / shelf space	Display size	Expected revenue from user history
Capital Budgeting / R&D	Fund project i?	Annual budget ($)	Project cost	Net Present Value, NPV_i
Digital Ad Auctions	Show ad i in feed?	Ad break / slot limit	Ad duration / pixels	Expected ad click revenue
Cargo & Freight Loading	Load container i?	Plane/truck payload	Container weight	Freight fee
Sports Salary Cap (GM)	Sign player i?	Team salary cap ($)	Player contract	Expected wins added

A Simple Knapsack Example

Which Items Should We Take?

Let's define a specific problem:

Knapsack Capacity

Maximum Weight (W) = **10 kg**

Available Items ($n = 4$)

Item (i)	**Name**	**Value (v_i)**	**Weight (w_i)**
1	Laptop	\$40	4 kg
2	Gold	\$50	3 kg
3	Silver	\$30	6 kg
4	Books	\$10	5 kg

The Question

How do we formally model the decision to maximize total value without exceeding 10 kg?

ILP Formulation

Translating the Problem into Math

We can model this problem using Binary Integer Linear Programming:

1. Decision Variables

- For each item i, define a binary variable $x_i \in \{0, 1\}$:
- $x_i = 1$ if we **take** item i
- $x_i = 0$ if we **leave** item i

x_1: Laptop, x_2: Gold, x_3: Silver, x_4: Books

2. Objective Function (Maximize Value)

Maximize the total value of selected items:

$$\max z = \sum_{i=1}^{4} v_i x_i = 40x_1 + 50x_2 + 30x_3 + 10x_4$$

3. Constraint (Weight Capacity)

Total weight cannot exceed $W = 10$:

$$4x_1 + 3x_2 + 6x_3 + 5x_4 \leq 10$$

How Does a "Greedy" Heuristic Work?

- Rank items by highest "bang for the buck" (value per kg, v_i/w_i):
  - **1. Gold:** \$50/3 kg = \$16.67/kg
  - **2. Laptop:** \$40/4 kg = \$10.00/kg
  - **3. Silver:** \$30/6 kg = \$5.00/kg
  - **4. Books:** \$10/5 kg = \$2.00/kg
- **Greedy Step 1:** Take Gold ($x_2 = 1$). *Value = \$50, Weight = 3 kg (7 kg left).*
- **Greedy Step 2:** Take Laptop ($x_1 = 1$). *Value* = \$50 + \$40 = **\$90**, *Weight* $= 3 + 4 = 7$ *kg (3 kg left).*
- Neither Silver (6 kg) nor Books (5 kg) fit in the remaining 3 kg.
- **Greedy Candidate:** {Laptop, Gold} with $z =$ \$90.

The Complete 0-1 Knapsack ILP Model

$$\begin{aligned} \max \quad & z = 40x_1 + 50x_2 + 30x_3 + 10x_4 \\ \text{s.t.} \quad & 4x_1 + 3x_2 + 6x_3 + 5x_4 \leq 10 \\ & x_1, x_2, x_3, x_4 \in \{0, 1\} \end{aligned}$$

Optimal ILP Solution

- Solving the exact ILP confirms that the greedy candidate **is** optimal in this instance:
- $x_1^* = 1,\ x_2^* = 1,\ x_3^* = 0,\ x_4^* = 0$
- Optimal Value: $z^* = 40(1) + 50(1) + 0 + 0 = \mathbf{\$90}$
- Total Weight: $w = 4(1) + 3(1) = 7 \text{ kg} \leq 10 \text{ kg}$
- *Comparison:* {Gold, Silver} uses more capacity ($w = 9$ kg) but yields lower value ($z = \$80$).

Reflecting on the Knapsack Solution

- Greedy heuristics often **fail** for 0-1 Knapsack problems (even though it happened to match the optimum here).
- *Counterexample:* If Silver weighed 7 kg with value \$45 ($v_3/w_3 = \$6.43/\text{kg}$), Greedy would still pick {Gold, Laptop} ($z = \$90, w = 7$), whereas {Gold, Silver} would achieve $z = \$95, w = 10$!
- The ILP formulation guarantees the true global optimum.
- For large n, Simplex alone cannot solve 0-1 Knapsack; it is solved via Branch-and-Bound.

Solving the 0-1 Knapsack in Python (PuLP 4.0)

Example 2: Binary Decision Variables with `add_variable_dicts`

```python
import pulp

items = ["Laptop", "Gold", "Silver", "Books"]
val = {"Laptop": 40, "Gold": 50, "Silver": 30, "Books": 10}
wt  = {"Laptop": 4,  "Gold": 3,  "Silver": 6,  "Books": 5}

prob = pulp.LpProblem("Knapsack_0_1", pulp.LpMaximize)

# Binary variables x[i] in {0, 1} using PuLP 4.0 API
x = prob.add_variable_dicts("x", items, cat=pulp.LpBinary)

# Objective & Weight Capacity Constraint
prob += pulp.lpSum(val[i] * x[i] for i in items), "Total_Value"
prob += pulp.lpSum(wt[i] * x[i] for i in items) <= 10, "Max_Weight"

res = prob.solve()
z_opt = pulp.value(prob.objective)
print(f"Status: {res.status.name}, Optimal Value = ${z_opt:.0f}")
for i in items:
    if x[i].varValue > 0.5:
        print(f"  Take {i}: weight={wt[i]} kg, value=${val[i]}")
```

Console Output

```
Status:  Optimal
Optimal Value = $90
Take Laptop:  weight=4 kg,
value=$40
Take Gold:  weight=3 kg, value=$50
```

Key Takeaway

`cat=pulp.LpBinary` restricts each variable to $\{0, 1\}$. Using `prob.add_variable_dicts` scales cleanly to hundreds of items.

Solving the 0-1 Knapsack in Google Sheets (OpenSolver)

Standard LP Spreadsheet Setup + Binary (`bin`) Variable Constraints

The spreadsheet setup is **identical to a standard LP**—the only difference is adding a **binary (`bin`)** constraint on the decision variables in OpenSolver.

1. Google Sheets Layout & Formulas

	A	B	C	D
1	**Item**	**Value**	**Weight**	**Take? (x_i)**
2	Laptop	40	4	**1**
3	Gold	50	3	**1**
4	Silver	30	6	**0**
5	Books	10	5	**0**
7	**Totals**	**90**	**7**	
8	**Limit**		**10**	

- **Objective Cell (`B7`):** `=SUMPRODUCT(B2:B5, D2:D5)`
- **Total Weight Cell (`C7`):** `=SUMPRODUCT(C2:C5, D2:D5)`
- **Max Capacity Cell (`C8`):** `10`

2. OpenSolver Steps

1. Open **Extensions** → **OpenSolver** → **Open Sidebar**.
2. **Objective Cell:** Set `B7` to **Max**.
3. **Variable Cells:** Set `D2:D5`.
4. **Weight Constraint:** Add `C7 <= C8` (Weight ≤ 10).
5. **Binary Constraint (Key Step!):** Add a constraint on `D2:D5` and choose `bin` (Binary 0/1) from the operator dropdown.
6. Click **Save Model**, then **Solve** ⇒ `B7 = $90`.

Section 3

The 0-1 Knapsack Problem: Logical Constraints

The Base Knapsack Problem (KP)

Standard Binary Integer Programming Model

Decision Variables

For each item i, let x_i be a **binary decision variable**:

$$x_i = \begin{cases} 1 & \text{if item } i \text{ is selected} \\ 0 & \text{otherwise} \end{cases}$$

Model Notation

- v_i: Value of item i
- w_i: Weight of item i
- C: Capacity of the knapsack

The Standard KP Model

$$\max \sum_{i \in I} v_i x_i$$

$$\text{s.t.} \quad \sum_{i \in I} w_i x_i \leq C$$

$$x_i \in \{0, 1\}, \quad \forall i \in I$$

- Next, we add **logical conditions** (implications, requirements, conflicts) using linear inequalities.

Complication 1: Conditional Inclusion

Rule: If Item 1 is Taken, Then Item 2 Must Be Taken

Constraint Rule: $x_1 = 1 \implies x_2 = 1$

This is a logical implication constraint. If the premise ($x_1 = 1$) is true, the conclusion ($x_2 = 1$) must also be true.

Complication 1: Conditional Inclusion

Rule: If Item 1 is Taken, Then Item 2 Must Be Taken

Constraint Rule: $x_1 = 1 \implies x_2 = 1$

This is a logical implication constraint. If the premise ($x_1 = 1$) is true, the conclusion ($x_2 = 1$) must also be true.

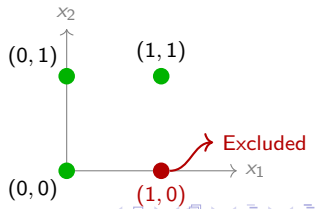
Test Cases (All Combinations)

- $x_1 = 1, x_2 = 0 \implies 1 - 0 = 1 \not\leq 0$ (**Violates** → Excluded)
- $x_1 = 1, x_2 = 1 \implies 1 - 1 = 0 \leq 0$ (**Satisfies** → Feasible)
- $x_1 = 0, x_2 = 0 \implies 0 - 0 = 0 \leq 0$ (**Satisfies** → Feasible)
- $x_1 = 0, x_2 = 1 \implies 0 - 1 = -1 \leq 0$ (**Satisfies** → Feasible)

The MILP Constraint

$$\mathbf{x_1 - x_2 \leq 0} \quad (\text{or } \mathbf{x_1 \leq x_2})$$

Visualization of Feasible Combinations



Complication 2: Mandatory Inclusion

Rule: Item 1 Must Be Taken

Constraint Rule: Item 1 is Mandatory

This is the simplest form of logical constraint, forcing a specific binary decision to be true regardless of other factors (as long as capacity allows).

Complication 2: Mandatory Inclusion

Rule: Item 1 Must Be Taken

Constraint Rule: Item 1 is Mandatory

This is the simplest form of logical constraint, forcing a specific binary decision to be true regardless of other factors (as long as capacity allows).

Impact on the Model

- All feasible solutions must now include Item 1.
- The objective and capacity constraints effectively become:

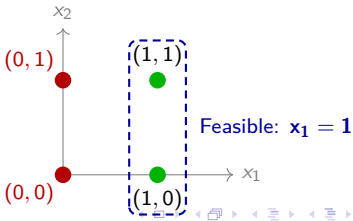
$$\max \left(v_1 + \sum_{i \in I \setminus \{1\}} v_i x_i \right)$$

$$\text{s.t.} \quad w_1 + \sum_{i \in I \setminus \{1\}} w_i x_i \leq C$$

The MILP Constraint

$$\mathbf{x_1 = 1}$$

Visualization: Forcing $x_1 = 1$



Complication 3: Mutual Exclusion (Prohibition)

Rule: If Item 1 is Taken, Then Item 2 Cannot Be Taken

Constraint Rule: $x_1 = 1 \implies x_2 = 0$

This relationship is symmetric for binary variables. If we take Item 1, we exclude Item 2, and if we take Item 2, we exclude Item 1. This is known as a **Mutual Exclusion** constraint.

Complication 3: Mutual Exclusion (Prohibition)

Rule: If Item 1 is Taken, Then Item 2 Cannot Be Taken

Constraint Rule: $x_1 = 1 \implies x_2 = 0$

This relationship is symmetric for binary variables. If we take Item 1, we exclude Item 2, and if we take Item 2, we exclude Item 1. This is known as a **Mutual Exclusion** constraint.

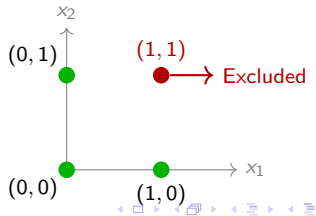
Test Cases (All Combinations)

- $x_1 = 1, x_2 = 1 \implies 1 + 1 = 2 \not\leq 1$ (**Violates** $\rightarrow$ Excluded)
- $x_1 = 1, x_2 = 0 \implies 1 + 0 = 1 \leq 1$ (**Satisfies** $\rightarrow$ Feasible)
- $x_1 = 0, x_2 = 1 \implies 0 + 1 = 1 \leq 1$ (**Satisfies** $\rightarrow$ Feasible)
- $x_1 = 0, x_2 = 0 \implies 0 + 0 = 0 \leq 1$ (**Satisfies** $\rightarrow$ Feasible)

The MILP Constraint

$$\mathbf{x_1 + x_2 \leq 1}$$

Visualization of Feasible Combinations



Complication 4: Exclusive OR (XOR)

Rule: Either Item 1 OR Item 2 Must Be Taken (But Not Both)

Constraint Rule: $(x_1 = 1 \text{ XOR } x_2 = 1)$

This constraint requires an exact choice of one item from the pair: **at least one** must be chosen, and **at most one** can be chosen.

Complication 4: Exclusive OR (XOR)

Rule: Either Item 1 OR Item 2 Must Be Taken (But Not Both)

Constraint Rule: ($x_1 = 1$ XOR $x_2 = 1$)

This constraint requires an exact choice of one item from the pair: **at least one** must be chosen, and **at most one** can be chosen.

Test Cases (All Combinations)

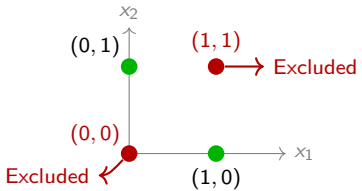
- $x_1 = 1, x_2 = 1 \implies 1 + 1 = 2 \neq 1$ (Excluded)
- $x_1 = 1, x_2 = 0 \implies 1 + 0 = 1$ (**Satisfies** $\rightarrow$ Feasible)
- $x_1 = 0, x_2 = 1 \implies 0 + 1 = 1$ (**Satisfies** $\rightarrow$ Feasible)
- $x_1 = 0, x_2 = 0 \implies 0 + 0 = 0 \neq 1$ (Excluded)

- Only the diagonal combinations $(\mathbf{1}, \mathbf{0})$ and $(\mathbf{0}, \mathbf{1})$ are allowed.

The MILP Constraint

$$\mathbf{x_1 + x_2 = 1}$$

Visualization of Feasible Combinations



Knapsack with Logical Constraints in Python (PuLP 4.0)

Example 3: Adding Mutual Exclusion and Implication Rules

```python
import pulp

items = ["Laptop", "Gold", "Silver", "Books"]
val = {"Laptop": 40, "Gold": 50, "Silver": 30, "Books": 10}
wt  = {"Laptop": 4,  "Gold": 3,  "Silver": 6,  "Books": 5}

prob = pulp.LpProblem("Knapsack_Logical", pulp.LpMaximize)
x = prob.add_variable_dicts("x", items, cat=pulp.LpBinary)

prob += pulp.lpSum(val[i] * x[i] for i in items), "Total_Value"
prob += pulp.lpSum(wt[i] * x[i] for i in items) <= 10, "Max_Weight"

# Rule 1 (Mutual Exclusion): Cannot take both Laptop and Gold
prob += x["Laptop"] + x["Gold"] <= 1, "Mutually_Exclusive"

# Rule 2 (Implication): If Gold is taken, Books must be taken
prob += x["Gold"] - x["Books"] <= 0, "Gold_Implies_Books"

res = prob.solve()
z_opt = pulp.value(prob.objective)
print(f"Status: {res.status.name}, Optimal Value = ${z_opt:.0f}")
print("Selected:", [i for i in items if x[i].varValue > 0.5])
```

Impact of Logical Rules

- **Option A (Take Gold):** Forces Books ($3 + 5 = 8$ kg), blocks Laptop; remaining 2 kg fits nothing
  $\Rightarrow z = 50 + 10 = \$60$.
- **Option B (Take Laptop + Silver):**
  $4 + 6 = 10$ kg ≤ 10 kg
  $\Rightarrow z = 40 + 30 = \mathbf{\$70}$.

Console Output

```
Status:  Optimal, Optimal Value = $70
Selected:  ['Laptop', 'Silver']
```

Section 4

Modeling Fixed Setup Costs

Problem Scenario: Capacity and Production

The Fixed Capacity Decision

A company manufactures two products, x_1 and x_2. They earn profits of **\$5 per unit of** x_1 and **\$3 per unit of** x_2.

The Fixed Cost Element

Before any production can occur (i.e., if $x_1 > 0$ or $x_2 > 0$), a single facility must be activated at a **fixed cost of \$10**.

Constraints (Resources)

- $\mathbf{C_1}$ (Resource A): $x_1 + x_2 \leq 10$
- $\mathbf{C_2}$ (Resource B): $2x_1 + x_2 \leq 16$
- $\mathbf{C_3}$ (Resource C): $x_2 \leq 7$

Decision Required

The final solution must choose between:

1. **No Production ($y = 0$):** Total Profit $Z = 0$.
2. **Production ($y = 1$):** Total Profit = (Gross Profit) − \$10.

Step 1: The Continuous LP Relaxation

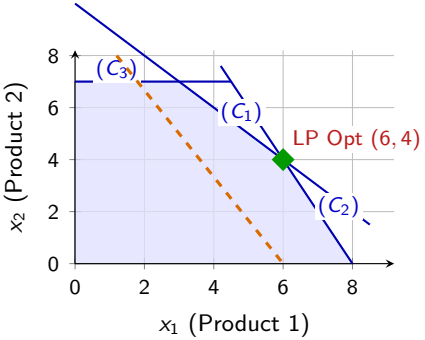
Solving for Maximum Gross Profit

We first maximize the gross profit $\mathbf{Z_{gross} = 5x_1 + 3x_2}$ while ignoring the fixed cost of \$10.

LP Formulation

$$\max Z_{\text{gross}} = 5x_1 + 3x_2$$

$$\begin{aligned} \text{s.t.} \quad & x_1 + x_2 \leq 10 && (C_1) \\ & 2x_1 + x_2 \leq 16 && (C_2) \\ & x_2 \leq 7 && (C_3) \\ & x_1, x_2 \geq 0 \end{aligned}$$



Result of LP Relaxation (Gross Profit)

The Gross Optimal Solution

- The optimal solution to the continuous Linear Program is found at the intersection of constraints $\mathbf{C_1}$ and $\mathbf{C_2}$:

$$\mathbf{C_1} : x_1 + x_2 = 10 \quad \Big| \quad \mathbf{C_2} : 2x_1 + x_2 = 16$$

- Solving these equations yields:

$$x_1^* = 6, \quad x_2^* = 4$$

- **Maximum Gross Profit (Z_{gross}):**

$$Z_{\text{gross}} = 5(6) + 3(4) = 30 + 12 = \$\mathbf{42}$$

If we use this LP solution, what is the NET Profit?

$$Z_{\text{net}} = Z_{\text{gross}} - \text{Fixed Cost} = 42 - 10 = \$\mathbf{32}$$

This establishes the maximum net profit if we decide to activate the facility.

Step 2: The Full MILP Formulation

Introducing the Binary Variable

We introduce a **binary variable** $y \in \{0, 1\}$ to enforce the fixed cost and link production levels to the activation decision.

Decision Variables

- $\mathbf{x_1, x_2} \in \mathbb{R}_{\geq \mathbf{0}}$ (Production quantities)
- $\mathbf{y} \in \{\mathbf{0}, \mathbf{1}\}$ (Facility activation indicator)

Objective Function (Net Profit)

The objective now includes the fixed cost, dependent on y:

$$\max Z = 5x_1 + 3x_2 - 10y$$

Fixed Cost Linking Constraint (Big-M)

We must link x_1, x_2 to y using a **Big-M** constraint:

- If $y = 0$, then $x_1 = x_2 = 0$.
- If $y = 1$, then x_1, x_2 are free to take feasible values.

Let $M = 10$ (tight upper bound on $x_1 + x_2$ from C_1):

$$\mathbf{x_1 + x_2 \leq 10y} \quad \Longleftrightarrow \quad \mathbf{x_1 + x_2 - 10y \leq 0}$$

The Complete Mixed-Integer Linear Program (MILP):

$$\max Z = 5x_1 + 3x_2 - 10y$$

$$\text{s.t.} \begin{cases} x_1 + x_2 \leq 10 & (C_1 : \text{Resource A}) \\ 2x_1 + x_2 \leq 16 & (C_2 : \text{Resource B}) \\ x_2 \leq 7 & (C_3 : \text{Resource C}) \\ \mathbf{x_1 + x_2 - 10y \leq 0} & (\text{Big-}M \text{ Linking Constraint}) \\ x_1, x_2 \geq 0, \quad \mathbf{y} \in \{\mathbf{0}, \mathbf{1}\} & \end{cases}$$

Step 3: Solving the MILP – Case 1

Case 1: Facility Not Activated ($y = 0$)

If the company chooses **not** to pay the fixed cost, then $y = 0$.

- **Fixed-cost constraint becomes:**

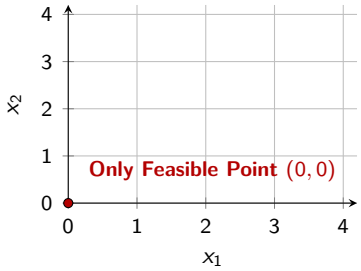
$$x_1 + x_2 - 10(0) \leq 0 \implies x_1 + x_2 \leq 0$$

- Since $x_1 \geq 0$ and $x_2 \geq 0$, the only feasible solution is:

$$\mathbf{x_1 = 0},\ \mathbf{x_2 = 0}.$$

- **Total profit:**

$$Z = 5(0) + 3(0) - 10(0) = \$\mathbf{0}.$$



Step 4: Solving the MILP – Case 2

Case 2: Facility Activated ($y = 1$)

If the company **pays** the fixed cost ($y = 1$), we use the LP optimal found in Step 1.

- **Fixed Cost Constraint becomes:**

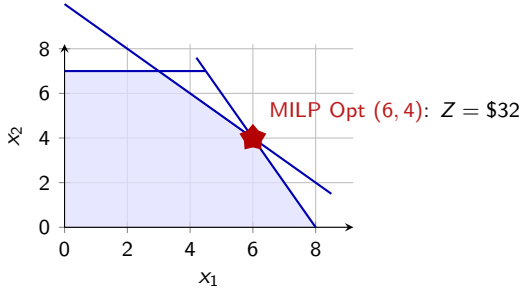
$$x_1 + x_2 - 10(1) \leq 0 \quad \Longrightarrow \quad x_1 + x_2 \leq 10$$

- This is identical to C_1, so the feasible region remains the original shaded polygon.
- The optimal production mix is the LP solution:

$$\mathbf{x_1^* = 6}, \quad \mathbf{x_2^* = 4}$$

- **Total Net Profit (Z):**

$$Z = 5(6) + 3(4) - 10(1) = 42 - 10 = \mathbf{\$32}$$



Conclusion: The Optimal MILP Solution

Comparing the Binary Choices

- **Case 1 (No Capacity):** $y = 0 \implies Z = \$0$
- **Case 2 (Capacity Activated):** $y = 1 \implies Z = \$32$

Optimal Decision

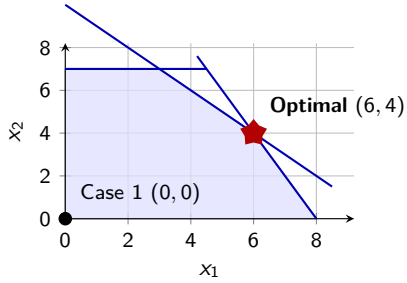
The optimal profit is $\mathbf{Z^* = \$32}$, achieved in Case 2.

Optimal MILP Solution

- $\mathbf{y^* = 1}$ (Activate facility)
- $\mathbf{x_1^* = 6}$ units, $\mathbf{x_2^* = 4}$ units

The fixed cost ($10) is outweighed by the gross profit ($42).

Visualization of the Two Candidate Optima



Fixed Setup Cost MILP in Python (PuLP 4.0)

Example 4: Modeling Big-M Linking Constraints

```python
import pulp

prob = pulp.LpProblem("Fixed_Setup_Cost_MILP", pulp.LpMaximize)

# Continuous production variables and binary setup variable
x1 = prob.add_variable("x1", lowBound=0, cat=pulp.LpContinuous)
x2 = prob.add_variable("x2", lowBound=0, cat=pulp.LpContinuous)
y  = prob.add_variable("y", cat=pulp.LpBinary)

# Objective: Gross Profit - Fixed Setup Cost
prob += 5 * x1 + 3 * x2 - 10 * y, "Net_Profit"

# Resource constraints + Big-M linking constraint (M = 10)
prob += x1 + x2 <= 10, "Resource_A"
prob += 2 * x1 + x2 <= 16, "Resource_B"
prob += x2 <= 7, "Resource_C"
prob += x1 + x2 - 10 * y <= 0, "Big_M_Link"

res = prob.solve()
z_opt = pulp.value(prob.objective)
print(f"Status: {res.status.name}, Net Profit Z* = ${z_opt:.2f}")
print(f"Activate y*={int(y.varValue)}, x1*={x1.varValue}, x2*={x2.varValue}")
```

Console Output

```
Status:  Optimal
Net Profit Z* = $32.00
Activate Facility y* = 1
x1* = 6.0, x2* = 4.0
```

Why Big-M Matters

Without `x1 + x2 - 10 * y <= 0`, the solver would set `y = 0` to avoid the \$10 fee while still producing `x1 = 6, x2 = 4`!

Section 5

Modeling a Min Purchase Quantity

Problem Description: Minimum Purchase Quantity

Context:

- A company can order two products — A and B — from a single supplier.
- Ordering from the supplier incurs a fixed setup (shipping) cost of \$12.
- If an order is placed, the supplier requires a **minimum purchase quantity (MOQ)** of 6 total units.
- Each product consumes limited assembly hours (16 hours available).
- The firm must decide whether to place an order at all, and how many units of each product to order.

Goal:

- Maximize total net profit = profit from A and B − fixed ordering cost.

Decision Variables and Parameters

Decision Variables:

- x_1 = quantity of Product A to order (units)
- x_2 = quantity of Product B to order (units)
- $y = \begin{cases} 1 & \text{if an order is placed,} \\ 0 & \text{otherwise.} \end{cases}$

Parameters:

- Profit per unit: \$7 for A, \$5 for B
- Fixed ordering cost: \$12 (incurred only if $y = 1$)
- Assembly time: 2 hours for A, 1 hour for B (max 16 hours)
- Minimum purchase quantity (MOQ): at least 6 total units if an order is placed ($y = 1$)

MILP with Minimum Purchase Quantity (MOQ)

Focusing on the Switching Constraints:

- Apart from the single resource limit ($2x_1 + x_2 \leq 16$), the core integer programming innovation is the pair of **switching constraints** controlled by $y \in \{0, 1\}$:

$$\max\ Z = 7x_1 + 5x_2 - 12y$$

$$\text{s.t.} \begin{cases} \text{Assembly hours:} & 2x_1 + x_2 \leq 16 \\ \textbf{MOQ switch (lower bound):} & \mathbf{x_1 + x_2 \geq 6\,y} \\ \textbf{Big-}M\textbf{ switch (upper bound):} & \mathbf{x_1 + x_2 \leq 16\,y} \\ x_1, x_2 \in \mathbb{Z}_{\geq 0}, \quad \mathbf{y \in \{0, 1\}} \end{cases}$$

Two-Sided Switching Rule: $\mathbf{6y \leq x_1 + x_2 \leq 16y}$

- If $y = 0$: $0 \leq x_1 + x_2 \leq 0 \implies x_1 = x_2 = 0$ (no purchase allowed).
- If $y = 1$: $6 \leq x_1 + x_2 \leq 16$ (MOQ enforced; $M = 16$ is the natural max from $2x_1 + x_2 \leq 16$).

Solving the MOQ Model: Optimal Solution & Analysis

Comparing No-Order ($y = 0$) vs. Active Order ($y = 1$)

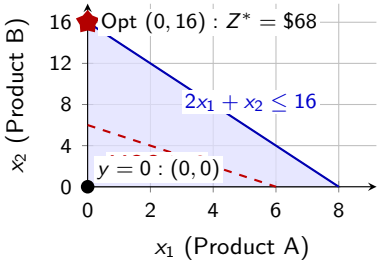
Case 1: Do Not Order ($y = 0$)

- Switching rule $0 \leq x_1 + x_2 \leq 0$ forces $x_1 = 0,\ x_2 = 0 \implies Z = \$\mathbf{0}$.

Case 2: Place Order ($y = 1$)

- Switching rule becomes $6 \leq x_1 + x_2 \leq 16$ alongside $2x_1 + x_2 \leq 16$.
- Profit per assembly hour: Product B earns \$5/hr vs. Product A at $\$7/2 = \$3.50$/hr.
- **Optimal Solution:** $x_1^* = 0,\ x_2^* = 16,\ y^* = 1$

$$Z^* = 7(0) + 5(16) - 12(1) = \$\mathbf{68.00}.$$



Solving the MOQ Model in Python (PuLP 4.0)

Example 5: Disjunctive Feasible Region with Switching Constraints

```python
import pulp

prob = pulp.LpProblem("Supplier_Order_MOQ", pulp.LpMaximize)

# Integer order quantities x1, x2 and binary order indicator y
x1 = prob.add_variable("x1", lowBound=0, cat=pulp.LpInteger)
x2 = prob.add_variable("x2", lowBound=0, cat=pulp.LpInteger)
y  = prob.add_variable("y", cat=pulp.LpBinary)

prob += 7 * x1 + 5 * x2 - 12 * y, "Net_Profit"

# Single resource constraint
prob += 2 * x1 + x2 <= 16, "Assembly_Hours"

# Two-sided switching constraints: 6*y <= x1 + x2 <= 16*y
prob += x1 + x2 - 6 * y >= 0, "MOQ_Lower_Switch"
prob += x1 + x2 - 16 * y <= 0, "Big_M_Upper_Switch"

res = prob.solve()
z_opt = pulp.value(prob.objective)
print(f"Status: {res.status.name}, Profit Z* = ${z_opt:.2f}")
print(f"y*={y.varValue:.0f}, x1*={x1.varValue:.0f}, x2*={x2.varValue:.0f}")
```

Console Output

```
Status:  Optimal
Profit Z* = $68.00
Order y* = 1
x1* = 0, x2* = 16
```

Modeling Insight

Combining $\mathbf{x_1 + x_2 \geq 6y}$ with $\mathbf{x_1 + x_2 \leq 16y}$ creates a *semi-continuous* switch: total quantity is either $\mathbf{0}$ ($y = 0$) or **between 6 and 16** ($y = 1$).

Section 6

Fixed-Charge Transshipment Problem

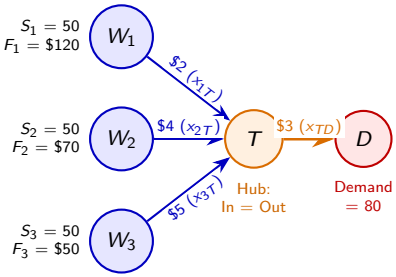
Fixed-Charge Transshipment Problem

Warehouse Selection and Network Flow

Logistics Network Setup:

- **3 Candidate Warehouses (W_1, W_2, W_3):** Each holds $S_i = 50$ units. Shipping any positive volume from W_i incurs a **fixed setup/dispatch fee** F_i.
- **Central Transshipment Hub (T):** Consolidates all inbound shipments (Flow In = Flow Out) and forwards them to Customer D at \$3/unit.
- **Customer Sink (D):** Requires exactly 80 units.

Warehouse	Supply (S_i)	Fixed Fee (F_i)	Cost (c_{iT})
W_1 (North)	50 units	\$120	\$2/unit
W_2 (Central)	50 units	\$70	\$4/unit
W_3 (South)	50 units	\$50	\$5/unit



Decision Variables and Warehouse Activation

Linking Continuous Network Flows to Binary Setup Choices

1. Continuous Flow Variables ($x_{ij} \geq 0$)

- $x_{1T}, x_{2T}, x_{3T} \geq 0$: Units shipped from Warehouse W_1, W_2, W_3 to Hub T.
- $x_{TD} \geq 0$: Consolidated units shipped from Hub T to Customer D.

2. Binary Activation Variables ($y_i \in \{0, 1\}$)

- $y_i = 1$ if Warehouse W_i is opened/used ($i = 1, 2, 3$), and $y_i = 0$ otherwise.

Big-M Capacity & Activation Rule

Each warehouse's inventory ($S_i = 50$) serves as the exact Big-M upper bound:

$$\mathbf{x_{iT} \leq 50 y_i} \iff \mathbf{x_{iT} - 50 y_i \leq 0} \quad (i = 1, 2, 3)$$

- $y_i = 0 \implies x_{iT} = 0$ (no flow without paying F_i).
- $y_i = 1 \implies x_{iT} \leq 50$ (full capacity unlocked).

Transshipment Hub Conservation (T)

Hub T holds no inventory (Flow In = Flow Out):

$$\mathbf{x_{1T} + x_{2T} + x_{3T} - x_{TD} = 0}$$

MILP Formulation: Fixed-Charge Transshipment

Combining Fixed Setup Fees, Big-M Capacities, and Flow Balance

$$\min \quad Z = \underbrace{120y_1 + 70y_2 + 50y_3}_{\text{Fixed Warehouse Setup Fees}} + \underbrace{2x_{1T} + 4x_{2T} + 5x_{3T} + 3x_{TD}}_{\text{Variable Transshipment Costs}}$$

Subject to:

$$x_{1T} - 50y_1 \leq 0 \quad \text{(Warehouse 1 Capacity \& Activation)}$$

$$x_{2T} - 50y_2 \leq 0 \quad \text{(Warehouse 2 Capacity \& Activation)}$$

$$x_{3T} - 50y_3 \leq 0 \quad \text{(Warehouse 3 Capacity \& Activation)}$$

$$x_{1T} + x_{2T} + x_{3T} - x_{TD} = 0 \quad \text{(Flow Conservation at Hub } T\text{)}$$

$$x_{TD} = 80 \quad \text{(Demand Requirement at Sink } D\text{)}$$

$$x_{1T}, x_{2T}, x_{3T}, x_{TD} \geq 0, \quad y_1, y_2, y_3 \in \{0, 1\}$$

Optimal Solution: Fixed vs. Variable Cost Trade-Off

Why the Cheapest Setup Fee Does Not Win

Since each warehouse holds 50 units and demand is 80 units, **at least two warehouses** must open:

1. Cheapest Setup Pair $\{W_2, W_3\}$ $(y_2 = 1, y_3 = 1)$

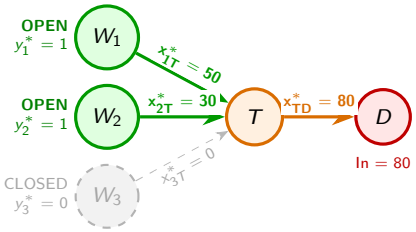
$x_{2T} = 50,\ x_{3T} = 30,\ x_{TD} = 80$
Fixed \$120 + Var \$590 $\Longrightarrow$ Total Cost = **\$710**.

2. Mixed Pair $\{W_1, W_3\}$ $(y_1 = 1, y_3 = 1)$

$x_{1T} = 50,\ x_{3T} = 30,\ x_{TD} = 80$
Fixed \$170 + Var \$490 $\Longrightarrow$ Total Cost = **\$660**.

3. Optimal MILP Pair $\{W_1, W_2\}$ $(y_1^* = 1, y_2^* = 1, y_3^* = 0)$

$x_{1T}^* = 50,\ x_{2T}^* = 30,\ x_{3T}^* = 0,\ x_{TD}^* = 80$
Fixed \$190 + Var \$460 $\Longrightarrow$ Optimal $\mathbf{Z}^*$ = **\$650**.



Economic Intuition

Opening W_1 costs \$70 more in fixed fees than W_3, but saves $(\$5 - \$2) \times 50 = \mathbf{\$150}$ in shipping—netting **\$60** over $\{W_2, W_3\}$!

Fixed-Charge Transshipment in Python (PuLP 4.0)

Example 6: Network Flow Conservation with Binary Setup Variables

```
import pulp

prob = pulp.LpProblem("Fixed_Charge_Transshipment", pulp.LpMinimize)
x1T = prob.add_variable("x1T", lowBound=0, cat=pulp.LpContinuous)
x2T = prob.add_variable("x2T", lowBound=0, cat=pulp.LpContinuous)
x3T = prob.add_variable("x3T", lowBound=0, cat=pulp.LpContinuous)
xTD = prob.add_variable("xTD", lowBound=0, cat=pulp.LpContinuous)
y1  = prob.add_variable("y1", cat=pulp.LpBinary)
y2  = prob.add_variable("y2", cat=pulp.LpBinary)
y3  = prob.add_variable("y3", cat=pulp.LpBinary)

prob += 120*y1 + 70*y2 + 50*y3 + 2*x1T + 4*x2T + 5*x3T + 3*xTD
prob += x1T - 50 * y1 <= 0, "Cap_W1"
prob += x2T - 50 * y2 <= 0, "Cap_W2"
prob += x3T - 50 * y3 <= 0, "Cap_W3"
prob += x1T + x2T + x3T - xTD == 0, "Hub_T_Balance"
prob += xTD == 80, "Sink_D_Demand"

res = prob.solve()
print(f"Status: {res.status.name}, Cost Z* = ${pulp.value(prob.objective):.2f}
      ")
print(f"Open: y1={y1.varValue:.0f}, y2={y2.varValue:.0f}, y3={y3.varValue:.0f}
      ")
print(f"Flow: x1T={x1T.varValue:.0f}, x2T={x2T.varValue:.0f}, xTD={xTD.
      varValue:.0f}")
```

Console Output

```
Status:  Optimal
Cost Z* = $650.00
Open:  y1=1, y2=1, y3=0
Flow:  x1T=50, x2T=30, xTD=80
```

LP Relaxation Trap

If y_i were relaxed to continuous $0 \leq y_i \leq 1$, the solver would set $y_2 = 30/50 = 0.6$ to pay only 60% of W_2's fixed fee ($Z_{\text{LP}} = \$622.00$). Enforcing `LpBinary` ensures full setup fees are paid!

Section 7

Store Location Problem (Set Covering)

Store Location on a City Grid

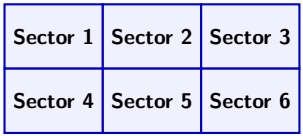
The Set Covering Problem

Retail Network Coverage Problem:

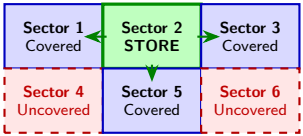
- A retail chain is entering a city divided into a 2×3 **grid of 6 sectors** $(1, \ldots, 6)$.
- Opening a store in Sector j serves customers in **Sector j itself** plus all **orthogonally adjacent sectors** (sharing a horizontal or vertical border; diagonals do not count).
- **Goal:** Open the **minimum number of stores** so every sector is covered by *at least one* store.

Sector i	**Adjacent Neighborhood $N(i)$ (Can Cover i)**
Sector 1	$\{1, 2, 4\}$
Sector 2	$\{1, 2, 3, 5\}$
Sector 3	$\{2, 3, 6\}$
Sector 4	$\{1, 4, 5\}$
Sector 5	$\{2, 4, 5, 6\}$
Sector 6	$\{3, 5, 6\}$

City Grid: 6 Sectors (2×3)



Example: Store in Sector 2 ($x_2 = 1$)



Binary Decision Variables and Coverage Logic

Translating Grid Adjacency into Linear Inequalities

1. Binary Location Variables ($x_j \in \{0, 1\}$)

For each grid sector $j \in \{1, 2, 3, 4, 5, 6\}$:

$$x_j = \begin{cases} 1 & \text{if a store is opened in Sector } j, \\ 0 & \text{otherwise.} \end{cases}$$

2. Objective Function

Minimize the total number of stores opened across the city:

$$\min \quad Z = \sum_{j=1}^{6} x_j = x_1 + x_2 + x_3 + x_4 + x_5 + x_6$$

3. Set Covering Constraint Rule

Every sector $i \in \{1, \ldots, 6\}$ must have **at least one** open store within its neighborhood $N(i)$:

$$\sum_{\mathbf{j \in N(i)}} \mathbf{x_j} \geq \mathbf{1} \quad \forall \mathbf{i} \in \{\mathbf{1}, \ldots, \mathbf{6}\}$$

Examples of Sector Coverage Constraints

- **Sector 1** ($N(1) = \{1, 2, 4\}$):

$$x_1 + x_2 + x_4 \geq 1$$

- **Sector 2** ($N(2) = \{1, 2, 3, 5\}$):

$$x_1 + x_2 + x_3 + x_5 \geq 1$$

Complete 0-1 ILP Formulation: Grid Store Location

Set Covering Model for All Six Sectors

$$\min \quad Z = x_1 + x_2 + x_3 + x_4 + x_5 + x_6$$

Subject to (One Coverage Constraint per Sector):

$$\begin{aligned}
x_1 + x_2 \qquad\quad + x_4 \qquad\quad &\geq 1 \quad \text{(Cover Sector 1: } \{1,2,4\}) \\
x_1 + x_2 + x_3 \qquad\quad + x_5 \qquad &\geq 1 \quad \text{(Cover Sector 2: } \{1,2,3,5\}) \\
x_2 \quad + x_3 \qquad\quad\quad + x_6 \quad &\geq 1 \quad \text{(Cover Sector 3: } \{2,3,6\}) \\
x_1 \qquad + x_4 + x_5 \qquad\quad &\geq 1 \quad \text{(Cover Sector 4: } \{1,4,5\}) \\
x_2 \qquad + x_4 + x_5 + x_6 \quad &\geq 1 \quad \text{(Cover Sector 5: } \{2,4,5,6\}) \\
x_3 \qquad\quad + x_5 + x_6 \quad &\geq 1 \quad \text{(Cover Sector 6: } \{3,5,6\})
\end{aligned}$$

$$x_1, x_2, x_3, x_4, x_5, x_6 \in \{0, 1\}$$

Why ≥ 1 Instead of $= 1$?

Using ≥ 1 (*Set Covering*) allows overlapping coverage when convenient; forcing $= 1$ (*Exact Set Partitioning*) would forbid a sector from bordering two open stores!

Optimal Solution: Covering the Grid with Two Stores

Analyzing Optimal Store Placements ($Z^* = 2$)

Why 1 Store Is Impossible ($Z \geq 2$)

Central Sectors 2 and 5 cover $|N(i)| = 4$ sectors each. Since $4 < 6$, **no single store can cover all 6 sectors**.

Optimal Solution A: Central Spine $\{2, 5\}$

- Set $\mathbf{x_2^* = 1}$, $\mathbf{x_5^* = 1}$ (all other $x_j^* = 0 \implies \mathbf{Z^* = 2}$).
- Store 2 covers $\{1, 2, 3, 5\}$; Store 5 covers $\{2, 4, 5, 6\}$.
- Union $= \{1, \ldots, 6\}$ (Sectors 2 & 5 have *double coverage*).

Optimal Solution B: Opposite Corners $\{1, 6\}$ or $\{3, 4\}$

- Set $\mathbf{x_1^* = 1}$, $\mathbf{x_6^* = 1}$ (or $\mathbf{x_3^* = 1}$, $\mathbf{x_4^* = 1}$) $\implies \mathbf{Z^* = 2}$.
- Store 1 covers $\{1, 2, 4\}$; Store 6 covers $\{3, 5, 6\}$ (zero overlap).

Optimum A: Central Spine ($x_2^* = 1, x_5^* = 1$)

Sector 1 (by S_2)	Sector 2 **STORE**	Sector 3 (by S_2)
Sector 4 (by S_5)	Sector 5 **STORE**	Sector 6 (by S_5)

Optimum B: Opposite Corners ($x_1^* = 1, x_6^* = 1$)

Sector 1 **STORE**	Sector 2 (by S_1)	Sector 3 (by S_6)
Sector 4 (by S_1)	Sector 5 (by S_6)	Sector 6 **STORE**

Store Location Set Covering in Python (PuLP 4.0)

Example 7: Dictionary-Driven Neighborhood Coverage Model

```python
import pulp

prob = pulp.LpProblem("City_Grid_Set_Covering", pulp.LpMinimize)

sectors = [1, 2, 3, 4, 5, 6]
neighbors = {
    1: [1, 2, 4],    2: [1, 2, 3, 5], 3: [2, 3, 6],
    4: [1, 4, 5],    5: [2, 4, 5, 6], 6: [3, 5, 6],
}

# Binary store location variables x[1] .. x[6]
x = {j: prob.add_variable(f"x_{j}", cat=pulp.LpBinary) for j in sectors}

# Objective: Minimize total number of stores opened
prob += sum(x[j] for j in sectors), "Total_Stores"

# Coverage constraint for each sector i
for i in sectors:
    prob += sum(x[j] for j in neighbors[i]) >= 1, f"Cover_Sector_{i}"

res = prob.solve()
chosen = [j for j in sectors if x[j].varValue > 0.5]
print(f"Status: {res.status.name}, Min Stores Z* = {pulp.value(prob.objective)
      :.0f}")
print(f"Open Stores in Sectors: {chosen}")
```

Console Output

```
Status:  Optimal
Min Stores Z* = 2
Open Stores in Sectors:
[3, 4] (or [2, 5] / [1, 6])
```

Modeling Scalability

By storing adjacency lists in a Python dictionary `neighbors[i]`, the exact same `for`-loop scales effortlessly from a 6-sector grid to a 500-zip-code metropolitan network!

Section 8

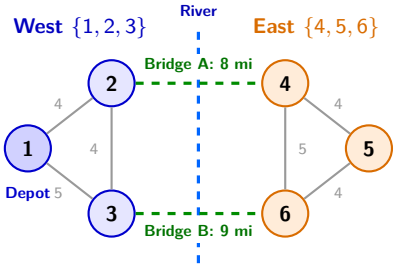
Last-Mile Delivery Routing (The Subtour Trap)

Last-Mile Delivery Routing

The Traveling Salesperson Problem (TSP) Across Two City Clusters

E-Commerce Van Routing Problem:

- A delivery van starts at **Depot 1** and must visit **5 customer stops** ($\{2, 3, 4, 5, 6\}$) and return to Depot 1 in a **single connected tour**.
- The city is split by a river into a **West Cluster** $\{1, 2, 3\}$ and an **East Cluster** $\{4, 5, 6\}$:
  - **Within West** $\{1, 2, 3\}$**:** $c_{12} = 4,\ c_{23} = 4,\ c_{13} = 5$ mi.
  - **Within East** $\{4, 5, 6\}$**:** $c_{45} = 4,\ c_{56} = 4,\ c_{46} = 5$ mi.
  - **River Bridges:** Bridge A ($c_{24} = 8$ mi) and Bridge B ($c_{36} = 9$ mi); all other cross-river links cost 12–16 mi.
- **Goal:** Minimize total mileage driven to visit all 6 stops in one continuous loop.



Binary Routing Variables & Degree Constraints

Formulating Arrival and Departure Rules at Every Stop

1. Binary Directed Arc Variables ($x_{ij} \in \{0, 1\}$)

For every ordered pair of distinct stops (i, j) with $i \neq j$:

$$x_{ij} = \begin{cases} 1 & \text{if the van drives directly from Stop } i \text{ to Stop } j, \\ 0 & \text{otherwise.} \end{cases}$$

2. Objective Function

Minimize total mileage across all selected road links:

$$\min \quad Z = \sum_{i=1}^{6} \sum_{j \neq i} c_{ij} x_{ij}$$

3. Out-Degree & In-Degree Constraints

- **Leave Each Stop Exactly Once (Out-Degree $= 1$):**

$$\sum_{j \neq i} x_{ij} = 1 \quad \forall i \in \{1, \ldots, 6\}$$

- **Enter Each Stop Exactly Once (In-Degree $= 1$):**

$$\sum_{i \neq j} x_{ij} = 1 \quad \forall j \in \{1, \ldots, 6\}$$

Critical Modeling Question

If every stop has **1 incoming arrow** and **1 outgoing arrow**, does that guarantee a single 6-stop tour? **No!** Let's see how the solver cheats.

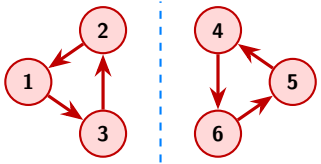
The "Subtour Trap": Why Degree Constraints Alone Fail!

Visualizing Disconnected Mini-Loops vs. a True Single Tour

Naive Degree Model ($Z_{\text{naive}} = 26$ mi — INVALID)

To avoid paying 8 mi and 9 mi to cross the river, the solver forms **two separate 3-stop subtours**:

- **West Subtour:** $1 \to 3 \to 2 \to 1$ $(5 + 4 + 4 = 13$ mi$)$
- **East Subtour:** $4 \to 6 \to 5 \to 4$ $(5 + 4 + 4 = 13$ mi$)$

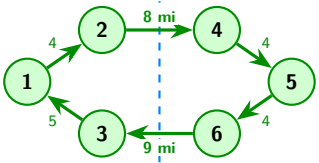


2 Disconnected Loops! Van never crosses river!

Subtour-Eliminated Tour ($Z^* = 34$ mi — VALID)

Adding **Subtour Elimination** forces the van across Bridge A $(2 \to 4)$ and back via Bridge B $(6 \to 3)$:

- **Single Tour:** $\mathbf{1 \to 2 \to 4 \to 5 \to 6 \to 3 \to 1}$
- **Mileage:** $4 + 8 + 4 + 4 + 9 + 5 = \mathbf{34}$ miles.



Single Connected 6-Stop Tour ($Z^* = 34$ mi)

Breaking the Loop: Intuition for Subtour Elimination

The "At Most $k-1$ Roads" Rule for Any Group of k Stops

The Core Intuition: How Do You Prevent a Closed Mini-Loop?

To form a closed loop among k stops, the van must use k **roads** inside that group. Therefore, if we cap the number of roads used inside any group of k stops at **at most** $k-1$, the van is **forced to drive out** to another stop!

1. Two-Stop & Three-Stop Groups

- **Any 2 Stops (At Most $2-1=1$ Road):**
  Cannot drive $1 \to 2 \to 1$ (15 equations):

$$x_{12} + x_{21} \leq 1, \quad x_{13} + x_{31} \leq 1, \ \ldots$$

- **Any 3 Stops (At Most $3-1=2$ Roads):**
  Cannot close a 3-stop loop like $\{1,2,3\}$ (20 eqs):

$$x_{12} + x_{21} + x_{13} + x_{31} + x_{23} + x_{32} \leq 2$$

2. Four-Stop & Five-Stop Groups

- **Any 4 Stops (At Most $4-1=3$ Roads):**
  Cannot close a 4-stop loop like $\{1,2,3,4\}$ (15 eqs):

$$(\text{Sum of roads inside } \{1,2,3,4\}) \leq 3$$

- **Any 5 Stops (At Most $5-1=4$ Roads):**
  Cannot close a 5-stop loop like $\{1,2,3,4,5\}$ (6 eqs):

$$(\text{Sum of roads inside } \{1,2,3,4,5\}) \leq 4$$

Complete 0–1 ILP Formulation: Last-Mile Delivery Routing

Putting the Entire 6-Stop Model Together—and Why Large Problems Explode!

Complete 6-Stop Routing ILP ($x_{ij} \in \{0,1\}$)

Objective (Minimize Mileage):

$\min\ Z = 4x_{12} + 4x_{21} + 5x_{13} + \cdots + 8x_{24} + 9x_{63} + \cdots + 4x_{56}$

1. Leave Each Stop Once (6 Equations):

$x_{12} + x_{13} + x_{14} + x_{15} + x_{16} = 1, \quad \ldots, \quad \sum_{j \neq 6} x_{6j} = 1$

2. Enter Each Stop Once (6 Equations):

$x_{21} + x_{31} + x_{41} + x_{51} + x_{61} = 1, \quad \ldots, \quad \sum_{i \neq 6} x_{i6} = 1$

3. Subtour Elimination (56 Equations Total):

- **15 Two-Stop Pairs:** $x_{12} + x_{21} \leq 1, \ \ldots$
- **20 Three-Stop Groups:**
  $x_{12} + x_{21} + x_{13} + x_{31} + x_{23} + x_{32} \leq 2, \ \ldots$
- **15 Four-Stop Groups:** (roads inside any 4 stops) ≤ 3
- **6 Five-Stop Groups:** (roads inside any 5 stops) ≤ 4

Why Large Routing Models Explode

Look at how fast the number of **Subtour Elimination Equations** grows as we add more delivery stops (n):

Delivery Stops (n)	Subtour Equations
6 stops (Our Example)	**56** equations
10 stops	**1,012** equations
20 stops	**1,048,554** equations
50 stops (1 Amazon Van)	$> \mathbf{10^{15}}$ (1 Quadrillion!)

Key Takeaway: Writing out all 2^n subtour constraints works great for small problems ($n \leq 15$), but real-world delivery routing with 50+ stops pushes even supercomputers to their limits!

Last-Mile Delivery Routing in Python (PuLP 4.0)

Example 8: Eliminating All 2-, 3-, 4-, and 5-Stop Subtours via combinations

```
from itertools import combinations
import pulp

nodes = [1, 2, 3, 4, 5, 6]
dist = {(1,2): 4, (1,3): 5, (2,3): 4, (4,5): 4, (4,6): 5, (5,6): 4,
        (2,4): 8, (3,6): 9, (1,4): 14, (1,5): 16, (1,6): 14,
        (2,5): 12, (2,6): 13, (3,4): 13, (3,5): 12}
c = {(i,j): dist.get((i,j), dist.get((j,i))) for i in nodes for j in nodes if i
      != j}

prob = pulp.LpProblem("Last_Mile_TSP", pulp.LpMinimize)
x = {(i,j): prob.add_variable(f"x_{i}_{j}", cat=pulp.LpBinary) for i, j in c}
prob += pulp.lpSum(c[i,j] * x[i,j] for i, j in c), "Total_Miles"
for i in nodes:
    prob += pulp.lpSum(x[i,j] for j in nodes if j != i) == 1, f"Out_{i}"
    prob += pulp.lpSum(x[j,i] for j in nodes if j != i) == 1, f"In_{i}"

# Eliminate subtours on ALL subsets S of size k = 2, 3, 4, 5 (2 <= |S| <= n-1)
for k in range(2, len(nodes)):
    for S in combinations(nodes, k):
        prob += pulp.lpSum(x[i,j] for i in S for j in S if i != j) <= k - 1

prob.solve()
print(f"Optimal Tour Mileage Z* = {pulp.value(prob.objective):.0f} miles")
```

Console Output

```
Without SEC (Naive):
Z = 26 miles (INVALID)
Loops:  1->3->2->1 & 4->6->5->4
With All Subtours Eliminated:
Optimal Mileage Z* = 34 miles
Valid Single Tour:
1 -> 2 -> 4 -> 5 -> 6 -> 3 -> 1
```

Why combinations Is Clean

Looping `k in range(2, len(nodes))` checks all $15 + 20 + 15 + 6 = 56$ proper subsets of size $2, 3, 4, 5$, guaranteeing **no subtour of any size** can form!

What When $n = 50+$ Stops? A Fast Routing Heuristic

Hand-Waving a Practical Two-Step Workaround: Nearest Neighbor + 2-Opt Uncrossing

1. Greedy "Nearest Neighbor" (39 mi)

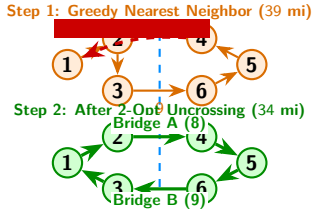
Skip the 2^n subtour equations and build one tour on the fly:

- **Rule:** From Depot 1, always drive to the **closest unvisited stop** next: $1 \xrightarrow{4} 2 \xrightarrow{4} 3 \xrightarrow{9} 6 \xrightarrow{4} 5 \xrightarrow{4} 4 \xrightarrow{14} 1$
- **Result: 39** mi *(valid tour, but stuck with costly* $4 \to 1$ *return!).*

2. "2-Opt" Uncrossing Swap (34 mi)

Spot any pair of tour links that **cross or overshoot**, delete them, and reconnect the two paths the other way:

- Swap $(2 \to 3,\ 4 \to 1)$ for Bridge A $(2 \to 4)$ and $(3 \to 1)$:
- **Result:** $\mathbf{1 \to 2 \to 4 \to 5 \to 6 \to 3 \to 1}$ **(34 mi — exact optimum!).**



Industry Rule of Thumb

Exact ILP ($n \leq 25$): 100% optimal. **Heuristics ($n = 100$–$10{,}000$):** Within 1–2% in < 0.1 sec (UPS ORION & Amazon)!

Section 9

Sports Analytics: Soccer Squad & Wage-Bill Optimization

Sports Analytics: 5-a-Side Soccer Squad Selection

Building a Squad Under a £500k/Week Wage Cap with 0–1 Integer Programming

Club Sporting Director & Recruitment Problem:

- A sporting director is signing a **5-a-side starting soccer squad** from a shortlist of 8 players under a **Weekly Wage Cap of £500,000/week** ($\leq$ £500k/wk).
- **Objective Metric (Season G+A):** Each player has an analytics projection for **Season Goal Contributions** (Goals + Assists, or Goals Prevented for goalkeepers).
- **Formation Quotas (2–2–1 Starting Five):**
  - **2 Midfielders (M)** from $\{P_1, P_2, P_3\}$
  - **2 Forwards (F)** from $\{P_4, P_5, P_6\}$
  - **1 Goalkeeper (GK)** from $\{P_7, P_8\}$
- **Goal:** Maximize total season Goal Contributions (Z) while obeying all weekly wage, formation, and tactical chemistry rules!

Shortlist (Wage Cap = £500k/wk)

ID	Player (Role)	Pos	League	G+A / Wage (£/w
P_1	De Bruyne (Playmaker)	M	EPL	26 G+A / £125k
P_2	Jude (Box-to-Box)	M	ESP	21 G+A / £95k
P_3	Rodri (Holding Mid)	M	EPL	14 G+A / £60k
P_4	Mbappé (Striker)	F	ESP	24 G+A / £115k
P_5	Vini (Left Wing)	F	ESP	17 G+A / £75k
P_6	Saka (Right Wing)	F	EPL	13 G+A / £55k
P_7	Ederson (Sweeper GK)	GK	EPL	18 GP / £80k
P_8	Courtois (Keeper)	GK	ESP	19 GP / £85k

Spicing Up the Squad: Dressing-Room & Tactical Logic Rules

Translating Soccer Tactics & Transfer Rules into Linear Inequalities

Rule 1: "Dressing-Room Feud" (Mutual Exclusion)

Vini (P_5) and Courtois (P_8) had a heated derby clash and refuse to play on the same squad (cannot sign both):

$$x_5 + x_8 \leq 1$$

Rule 2: "Build-From-The-Back Duo" (Conditional Stack)

Sweeper-keeper Ederson (P_7) only achieves 18 Goals Prevented/Created if playmaker De Bruyne (P_1) starts ahead of him to receive his long passes ($x_7 = 1 \implies x_1 = 1$):

$$x_7 \leq x_1 \iff x_7 - x_1 \leq 0$$

Rule 3: "Only One Penalty Taker" (Set-Piece Clash)

De Bruyne (P_1), Jude (P_2), and Mbappé (P_4) all demand penalties and free kicks; sign **at most 2** of these 3 superstars:

$$x_1 + x_2 + x_4 \leq 2$$

Rule 4: "Work-Permit / Club Quota" (Anti-Stacking Cap)

Transfer rules allow signing **at most 2 players from the same former club**. Since $\{P_1, P_2, P_7\}$ share **Club A**:

$$x_1 + x_2 + x_7 \leq 2$$

Complete 0–1 ILP Formulation: Soccer Squad Model

Combining the £500k/Week Wage Cap, Formation Quotas, and Tactical Rules

$$\max \quad Z = 26x_1 + 21x_2 + 14x_3 + 24x_4 + 17x_5 + 13x_6 + 18x_7 + 19x_8 \quad \text{(Total Season Goal Contributions)}$$

1. Weekly Wage Cap (£k/wk) & Formation Quotas:

$$\begin{aligned} 125x_1 + 95x_2 + 60x_3 + 115x_4 & \\ + 75x_5 + 55x_6 + 80x_7 + 85x_8 &\leq 500 \quad \text{(£500k/wk Cap)} \\ x_1 + x_2 + x_3 &= 2 \quad \text{(2 Midfielders)} \\ x_4 + x_5 + x_6 &= 2 \quad \text{(2 Forwards)} \\ x_7 + x_8 &= 1 \quad \text{(1 Goalkeeper)} \end{aligned}$$

2. Chemistry & Transfer Logical Rules:

$$\begin{aligned} x_5 + x_8 &\leq 1 \quad \text{(Feud: Vini / Courtois)} \\ x_7 - x_1 &\leq 0 \quad \text{(Stack: Ederson} \Rightarrow \text{De Bruyne)} \\ x_1 + x_2 + x_4 &\leq 2 \quad \text{(Max 2 Set-Piece Stars)} \\ x_1 + x_2 + x_7 &\leq 2 \quad \text{(Max 2 from Club A)} \\ x_1, \ldots, x_8 &\in \{0, 1\} \end{aligned}$$

How Tactical Rules Reshape the Optimal Soccer Squad

Comparing the Naive "Galácticos" Squad vs. the Chemistry-Compliant Optimum

1. Naive "Galácticos" Squad (No Logic Rules)

- **Midfielders:** P_1 De Bruyne (£125k/wk) + P_2 Jude (£95k/wk)
- **Forwards:** P_4 Mbappé (£115k/wk) + P_5 Vini (£75k/wk)
- **Goalkeeper:** P_8 Courtois (£85k/wk)
- **Stats:** Wage Bill = £495k/wk $\leq$ £500k, Goal Contrib = **107** goals.
- **Why It Blows Up:**
  1. Violates **Rule 1 (Feud)**: Signs both Vini and Courtois ($x_5 + x_8 = 2 > 1$).
  2. Violates **Rule 3 (Set-Piece Clash)**: Signs all 3 penalty/free-kick stars ($x_1 + x_2 + x_4 = 3 > 2$).

2. Chemistry-Optimal Squad ($Z^* = 99$ Goal Contributions)

- **Midfielders:** $\mathbf{P_1}$ De Bruyne (£125k) + $\mathbf{P_3}$ Rodri (£60k)
- **Forwards:** $\mathbf{P_4}$ Mbappé (£115k) + $\mathbf{P_5}$ Vini (£75k)
- **Goalkeeper:** $\mathbf{P_7}$ Ederson (£80k)
- **Stats:** Wage Bill = **£455**k/wk $\leq$ £500k, Goal Contrib = **99** goals.
- **How the Solver Adapts:**
  1. Keeps **De Bruyne + Mbappé** (2-star cap) and swaps Jude for holding midfielder **Rodri**.
  2. Swaps Courtois for **Ederson** to resolve the Vini feud ($x_5 + x_8 = 1$), unlocking the **De Bruyne–Ederson passing link** ($x_7 \leq x_1$) while keeping Club A at 2 players!

Soccer Squad Optimization in Python (PuLP 4.0)

Example 9: Maximizing Season Goal Contributions Under a £500k/Week Wage Cap

```
import pulp
players = ["P1_DeBruyne", "P2_Jude", "P3_Rodri", "P4_Mbappe",
           "P5_Vini", "P6_Saka", "P7_Ederson", "P8_Courtois"]
# Projected season Goal Contributions (G+A / Goals Prevented)
goals = {"P1_DeBruyne": 26, "P2_Jude": 21, "P3_Rodri": 14, "P4_Mbappe": 24,
         "P5_Vini": 17, "P6_Saka": 13, "P7_Ederson": 18, "P8_Courtois": 19}
# Weekly wages in thousands of pounds (k/week), Cap = 500k/week
wage  = {"P1_DeBruyne": 125, "P2_Jude": 95, "P3_Rodri": 60, "P4_Mbappe": 115,
         "P5_Vini": 75, "P6_Saka": 55, "P7_Ederson": 80, "P8_Courtois": 85}

prob = pulp.LpProblem("Soccer_Squad_ILP", pulp.LpMaximize)
x = prob.add_variable_dicts("x", players, cat=pulp.LpBinary)
prob += pulp.lpSum(goals[i] * x[i] for i in players), "Season_Goals"
prob += pulp.lpSum(wage[i] * x[i] for i in players) <= 500, "Wage_Cap_500k"
# Formation Quotas (2M, 2F, 1GK) & Tactical Chemistry Rules
prob += x["P1_DeBruyne"] + x["P2_Jude"] + x["P3_Rodri"] == 2, "Two_Mids"
prob += x["P4_Mbappe"] + x["P5_Vini"] + x["P6_Saka"] == 2, "Two_Fwds"
prob += x["P7_Ederson"] + x["P8_Courtois"] == 1, "One_GK"
prob += x["P5_Vini"] + x["P8_Courtois"] <= 1, "Feud_Exclusion"
prob += x["P7_Ederson"] - x["P1_DeBruyne"] <= 0, "BuildOut_Stack"
prob += x["P1_DeBruyne"] + x["P2_Jude"] + x["P4_Mbappe"] <= 2, "SetPiece_Cap"
prob += x["P1_DeBruyne"] + x["P2_Jude"] + x["P7_Ederson"] <= 2, "ClubA_Limit"
prob.solve()
squad = [p for p in players if x[p].varValue > 0.5]
print(f"Optimal G+A Z* = {pulp.value(prob.objective):.0f} | Squad = {squad}")
```

Console Output

```
Status:  Optimal
Optimal Goal Contrib Z* = 99
Weekly Wage Bill = £455k/wk
Starting 5 Soccer Squad:
 - P1_DeBruyne (M, £125k, 26)
 - P3_Rodri (M, £60k, 14)
 - P4_Mbappe (F, £115k, 24)
 - P5_Vini (F, £75k, 17)
 - P7_Ederson (GK, £80k, 18)
```

Why ILP Dominates Spreadsheets

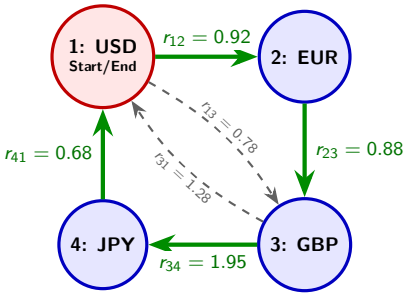
When scouting across 200+ transfer targets in Europe, there are trillions of valid squads—yet PuLP finds the exact global optimum in < 0.02 seconds!

Problem 10: Currency Arbitrage (Profitable FX Tours)

Exchanging Money Around a Multi-Currency Loop to Earn Risk-Free Profit

The FX Desk Challenge: You start with **$100,000** USD and observe 6 live exchange-rate quotes across **4 currencies**: **1: USD ($)**, **2: EUR (€)**, **3: GBP (£)**, and **4: JPY (100¥)**. Can you make a closed tour starting and ending in USD that multiplies your money (> 1.00)?

Arc	FX Conversion	Var	Rate r_{ij}	$\ln(r_{ij})$
$(1,2)$	USD $\to$ EUR	x_{12}	0.92	-0.0834
$(1,3)$	USD $\to$ GBP	x_{13}	0.78	-0.2485
$(2,3)$	EUR $\to$ GBP	x_{23}	0.88	-0.1278
$(3,1)$	GBP $\to$ USD	x_{31}	1.28	$+0.2469$
$(3,4)$	GBP $\to$ JPY	x_{34}	1.95	$+0.6678$
$(4,1)$	JPY $\to$ USD	x_{41}	0.68	-0.3857



Turning Multiplying Exchange Rates into a 0–1 Linear Program

The Logarithm Linearization Trick and Tour Flow Conservation

1. Why Products Look Nonlinear—and the Log Trick

Along any currency tour, exchange rates **multiply**:

$$\text{Multiplier} = \prod_{(i,j)} (r_{ij})^{x_{ij}} > 1.00$$

- Taking the natural logarithm (ln) turns a **product of rates** into a **linear sum of log-rates**:

$$\max \ \mathbf{Z} = \ln(\text{Multiplier}) = \sum_{(\mathbf{i},\mathbf{j})} \ln(\mathbf{r_{ij}})\, \mathbf{x_{ij}}$$

- **Arbitrage Rule:** Any tour with $Z^* > 0$ yields a compound multiplier $e^{Z^*} > 1.00$ (risk-free profit!).

2. Binary Variables & Tour Flow Balance

Let $x_{ij} \in \{0, 1\}$ equal 1 if our tour converts Currency $i \to j$:

- **Start & Finish in USD (Node 1):**

$$x_{12} + x_{13} = 1 \quad \text{(Leave USD)}$$

$$x_{31} + x_{41} = 1 \quad \text{(Return to USD)}$$

- **Intermediate Flow Conservation (In = Out):** Every foreign currency entered must also be exited:

$$\text{EUR (2): } x_{12} - x_{23} = 0$$

$$\text{GBP (3): } x_{13} + x_{23} - x_{31} - x_{34} = 0$$

$$\text{JPY (4): } x_{34} - x_{41} = 0$$

Complete 0–1 ILP & Comparing All Candidate FX Tours

Why the 4-Currency Tour Beats Every 2-Currency and 3-Currency Loop

Complete 6-Variable 0–1 Currency Arbitrage ILP

$$\max \; Z = -0.0834x_{12} - 0.2485x_{13} - 0.1278x_{23} + 0.2469x_{31} + 0.6678x_{34} - 0.3857x_{41}$$

$$\text{s.t.} \quad x_{12} + x_{13} = 1, \quad x_{31} + x_{41} = 1, \quad x_{12} - x_{23} = 0, \quad x_{13} + x_{23} - x_{31} - x_{34} = 0, \quad x_{34} - x_{41} = 0, \quad x_{ij} \in \{0, 1\}$$

Candidate Tour (Start & End in USD)	**Active Arcs ($x_{ij} = 1$)**	**Log Sum Z**	**Multiplier e^Z**	**P&L on \$100k**
1. Direct 2-Currency (USD → GBP → USD)	$x_{13}, x_{31} = 1$	-0.0016	$0.78 \times 1.28 = 0.9984$	$-\$160$ (Loss)
2. Asian Triangle (USD → GBP → JPY → USD)	$x_{13}, x_{34}, x_{41} = 1$	$+0.0337$	$0.78 \times 1.95 \times 0.68 = 1.0343$	$+\$3{,}428$ ($+3.43\%$)
3. Euro Triangle (USD → EUR → GBP → USD)	$x_{12}, x_{23}, x_{31} = 1$	$+0.0356$	$0.92 \times 0.88 \times 1.28 = 1.0363$	$+\$3{,}629$ ($+3.63\%$)
4. Global 4-Currency (USD → EUR → GBP → JPY → USD)	$\mathbf{x_{12}, x_{23}, x_{34}, x_{41} = 1}$	$\mathbf{+0.0710}$	$\mathbf{0.92 \times 0.88 \times 1.95 \times 0.68 = 1.0735}$	$\mathbf{+\$7{,}353}$ ($\mathbf{+7.35\%}$)

Currency Arbitrage in Python (PuLP 4.0)

Example 10: Finding the Most Profitable FX Loop via Log-Rates

```python
import math, pulp
rates = {
    ("USD", "EUR"): 0.92, ("USD", "GBP"): 0.78,
    ("EUR", "GBP"): 0.88, ("GBP", "USD"): 1.28,
    ("GBP", "JPY"): 1.95, ("JPY", "USD"): 0.68,
}
prob = pulp.LpProblem("Currency_Arbitrage_ILP", pulp.LpMaximize)
x = prob.add_variable_dicts("x", rates.keys(), cat=pulp.LpBinary)
# Maximize linear sum of log exchange rates: ln(r_ij) * x_ij
prob += pulp.lpSum(math.log(r) * x[a] for a, r in rates.items())
# Start and end at USD (Node 1)
prob += x[("USD", "EUR")] + x[("USD", "GBP")] == 1, "Leave_USD"
prob += x[("GBP", "USD")] + x[("JPY", "USD")] == 1, "Return_USD"
# Flow conservation (In == Out) at EUR, GBP, JPY
for c in ["EUR", "GBP", "JPY"]:
    inflow = pulp.lpSum(x[(i, j)] for (i, j) in rates if j == c)
    outflow = pulp.lpSum(x[(i, j)] for (i, j) in rates if i == c)
    prob += inflow - outflow == 0, f"Flow_{c}"
prob.solve()
mult = math.exp(pulp.value(prob.objective))
print(f"Multiplier = {mult:.4f} (+{(mult-1)*100:.2f}% profit)")
```

Console Output

```
Status:  Optimal
Optimal Log-Return Z* = +0.0710
Multiplier = 1.0735 (+7.35% profit)
Optimal FX Arbitrage Tour:
 1.  USD -> EUR (rate = 0.92)
 2.  EUR -> GBP (rate = 0.88)
 3.  GBP -> JPY (rate = 1.95)
 4.  JPY -> USD (rate = 0.68)
$100,000 -> $107,352.96 USD
```

Wall Street Takeaway

High-frequency FX trading desks run this exact log-rate cycle model in microseconds to lock in cross-currency mispricings before exchange rates adjust!

Conclusion

- **Solution Methods:** Simplex alone cannot solve ILPs; finding an optimal integer solution requires solving a sequence of LP relaxations via Branch-and-Bound and Cutting Planes.
- **Solver Ecosystem:** Industrial-scale commercial MILP solvers (e.g., Gurobi, CPLEX) are expensive, whereas open-source MILP solvers like **CBC** (bundled free with Python's `PuLP`) solve small-to-medium ILPs effectively.
- **Duality & Sensitivity:** Standard LP shadow prices and reduced costs do **not** directly apply to ILPs because discrete jumps break continuity.
- **Core Applications:** 0–1 selection problems (knapsack, set covering / store location, sports analytics salary-cap lineups), logical & chemistry conditions, fixed setup costs, minimum order quantities (MOQ), fixed-charge transshipment networks, last-mile delivery routing (TSP with subtour elimination), and currency arbitrage loops (via log-exchange-rates).